

Orbit Wars — v12 GRPO

The critic-free group-relative learner: the group rollout, the advantage-estimator family (mutually exclusive, with precedence), the clipped-surrogate update — and the empirical verdict that the anti-collapse estimators *failed* and legacy std-norm is the keeper

Source of truth: `orbit_wars_v12/rollout.py`, `grpo.py`, `grpo_diag.py`

2026-06-19

Abstract

This is document 3 of four. GRPO (`cfg.ALGO="grpo"`) is a *critic-free* PPO variant: it replaces the learned value baseline with a **group baseline** (several rollouts share one initial world; an advantage is the rollout’s return relative to its group). It assumes document 1 (`v12_setup.tex`) for the game, rollout, encoding, action, and reward, and reuses PPO’s clipped surrogate from document 2 verbatim. This document is precise about (i) how the group rollout is rendered, (ii) the family of advantage estimators and their *mutually exclusive* precedence, (iii) the update. **It also records the empirical result that matters most:** every “anti-collapse” estimator in that family caused passivity in controlled ablation, and the keeper is the *legacy* per-group-std baseline with symmetric outcome decay.

EMPIRICAL VERDICT — read before enabling any flag. The Tier 0–2 advantage-estimator family below (rank [C1], Dr.GRPO center-only [B], CVaR [C2], sibling [B1], phi-value [C], asymmetric clip [D3]) was built to stop a GRPO “reward collapse.” Controlled 3070 Ti ablations (`scripts/_gpu_ab*.py`, shared init) show **the estimator changes ARE what cause the collapse:** all of them drive the agent into passivity, while the **keeper is the legacy default** — `GRPO_STD_NORM=True`, every estimator flag *off*, `CLIP_HI=0` — which stays stable and improving. *Bounded $\neq$ good:* the rank bound held but discarded win-magnitude and stopped rewarding aggression. The shipped notebook `setup1_v12_grpo_a100.ipynb` is reverted to the keeper. Details in Section 5.

Contents

1	Why group-relative, and what changes vs PPO	2
2	The group rollout (<code>collect_ppo</code> , GRPO branch)	2
3	The advantage-estimator family	2
3.1	The flat estimators (<code>grpo_flat_advantage</code>)	3
3.2	The per-step estimators (critic-free GAE)	3
3.3	Orthogonal levers (these compose)	4
4	The update (<code>grpo_update</code>)	4
5	Empirical verdict: the estimators failed; legacy is the keeper	4

1 Why group-relative, and what changes vs PPO

PPO’s critic estimates a per-state baseline $V(s)$; GRPO instead rolls each world out G times and uses the *within-group* return statistics as the baseline, so the world’s intrinsic difficulty cancels without a learned value function. Concretely, versus `ppo_update`:

- **no value loss** — the critic head is left untrained and unused (unless the `GRPO_AUX_VALUE` representation `aux` is on);
- **no bootstrap** — there is no critic to bootstrap from at the horizon;
- **optional KL-to-reference** (DeepSeekMath $\beta = \text{GRPO_KL_COEF}$) against a frozen snapshot of the policy.

Everything else — the gated-allocation actor, the clipped surrogate, entropy bonus, KL early-stop, grad clipping, the GPU stat accumulators — is shared with PPO. PPO and GRPO checkpoints are interchangeable (the critic is simply dormant under GRPO). The advantage is precomputed in `collect_ppo` and stored in `tb["advantage"]`, exactly like PPO, so `grpо_update` just consumes it.

2 The group rollout (`collect_ppo`, GRPO branch)

When `cfg.ALGO="grpо"`, the rollout tiles n_W distinct worlds into contiguous groups of G envs (one group = one world rolled out G times), with $G = \text{GRPO_GROUP}$ or `GROUP_SIZE` (falling back to the whole batch if $B \bmod G \neq 0$). Each group is assigned a *single* seat geometry, cycling the seat rotations across groups so the ego still trains on every seat without breaking within-group homogeneity (which the baseline depends on). The per-group return is

$$R^e = \begin{cases} O^e / \text{PPO_REWARD_SCALE} & \text{GRPO_OUTCOME_ONLY (terminal outcome only),} \\ \sum_t r_t^e \text{ alive}_t^e & \text{otherwise (the full shaped return),} \end{cases}$$

reshaped to $\mathbb{R}^{n_W \times G}$ as `Rg`. **[D]** `GRPO_CRN` optionally shares the *opponent’s* sampling noise across the G rollouts of a group (common random numbers; the ego keeps independent noise) so the group baseline isolates ego-action variance.

3 The advantage-estimator family

The single most important structural fact: **the advantage estimators are mutually exclusive**. Turning several on does *not* stack them — the code silently selects exactly one by precedence. `grpо_adv_mode(cfg, n_players)` resolves the active estimator and the enabled-but-ignored ones, and `train.py` prints the active choice at start with a loud warning listing any ignored estimators. The precedence is

$$\text{phi-value [C]} \succ \text{sibling [B1]} \succ \text{analytic [A] (2p)} \succ \text{rank [C1]} \succ \text{CVaR [C2]} \succ \text{center/std [B]}.$$

This footgun is exactly why an “all flags on” run collapses on an *un-validated* per-step path (it runs sibling or phi-value, never rank). Orthogonal levers (next subsection) *do* compose.

3.1 The flat estimators (grpо_flat_advantage)

One scalar advantage per rollout, broadcast over its steps. With R^e as above:

- **[B] legacy / Dr.GRPO / RLOO (the default ladder)**. Centre on the group mean $\mu = \frac{1}{G} \sum_g R_g$ (or the leave-one-out mean if GRPO_LOO). Then GRPO_STD_NORM=True (default) divides by the per-group std, $A = (R - \mu) / (\sigma_g + \text{GRPO_ADV_EPS})$; GRPO_STD_NORM=False is Dr.GRPO — centre-only with *one global* scale (removes the cross-group difficulty-inversion bias of std-norm).
- **[A] analytic 2p binary** (GRPO_ANALYTIC_ADV, 2p only). At a Beta-shrunk win-rate $\hat{p} = (\#\text{wins} + \alpha_0) / (G + 2\alpha_0)$, the closed-form standardized advantage of a binary outcome: $A = \text{win} \sqrt{(1-\hat{p})/\hat{p}} - \text{loss} \sqrt{\hat{p}/(1-\hat{p})}$. *Not* anti-collapse — it has the same unbounded $\sqrt{(1-p)/p}$ as a rare win.
- **[C1] rank** (GRPO_RANK_ADV). Van der Waerden / NES average-rank score: with average rank $\in [0, G-1]$, $A = (\text{rank} - \frac{G-1}{2}) / \sqrt{(G^2-1)/12}$, bounded in $[-\sqrt{3}, \sqrt{3}]$ and invariant to every reward-magnitude knob.
- **[C2] CVaR** (GRPO_CVAR_ALPHA > 0). Risk-sensitive: baseline on the worst- α tail quantile, with extra gradient weight GRPO_CVAR_LAMBDA on the tail rollouts.

[A], [C1], and [B]-with-std are already $\sim$ unit-variance and are passed through unscaled; [B]-center-only and [C2] get one global scale (or ADV_TARGET_RMS, below).

3.2 The per-step estimators (critic-free GAE)

- **[B1] sibling** (GRPO_SIBLING_BASELINE, _grpо_sibling_advantage). The G shared-root rollouts of a group are bucketed by quantized $\Phi_{\text{ship}}(s_t)$ at each step; the matched-bucket mean terminal return is a per-step value $\hat{V}(s_t)$ (whole-group mean when a bucket is thinner than GRPO_SIBLING_MIN_PEERS); then GAE(γ, λ) on the per-step reward. A VinePPO-style per-step credit with no critic.
- **[C] phi-value** (GRPO_PHI_VALUE, needs USE_POTENTIAL_SHAPING). Build a value $\hat{V}(s_t) = \mu_g + a(\Phi(s_t) - \Phi(s_0))$ where μ_g is the group (or LOO) baseline and a is the OLS slope of the outcome on the terminal $\Delta\Phi$, clamped to $[0, \text{GRPO_PHI_A_MAX}]$; then GAE on the per-step reward. Here Φ is moved *out* of the reward (dense_t=0) and *into* the value, turning shaping (which the group mean would cancel) into a per-step baseline.

3.3 Orthogonal levers (these compose)

flag	tag	effect
CLIP_HI	D3	asymmetric upper clip in the shared surrogate (anti-passivity)
RATIO_LENGTHNORM	D1	GSPO geometric-mean ratio (/ #owned)
GRPO_RB_GATE	A1	Rao-Blackwellize the WHERE term over the launch Bernoulli (de-noise)
ADV_TARGET_RMS	D4	pin the (center-only) advantage RMS so league difficulty can't drift the effective LR
GRPO_OPP_BASELINE_W	E1	subtract the Elo-expected outcome vs this foe (a no-op under one-opponent-per-iter centering)
GRPO_AUX_VALUE	B2	train the dormant value head by MC regression as a representation aux (not in the advantage)
GRPO_LOO	—	leave-one-out group mean (RLOO)
GRPO_KL_COEF	—	KL-to-reference penalty β
GRPO_CRN	D	common-random-numbers opponent coupling

[E2] `GRPO_OPP_STRATA` is reserved (not wired): it would sample several opponents per iteration so the baseline averages opponent strength, at which point E1 stops being a no-op.

4 The update (`grpо_update`)

`grpо_update` runs the same `UPDATE_EPOCHS×MINIBATCHES` clipped-surrogate passes as PPO (document 2: the ratio, `CLIP/CLIP_HI`, per-owned-source entropy bonus, k3 KL early-stop, grad clip, single end-of-update host sync), with four differences:

- **no value loss:** $L = L^{\text{pol}} - \text{ent_coef} \bar{H}$ (the policy is always at full weight — `policy_coef` is accepted for call-site parity but there is no critic to warm up);
- **KL-to-reference** (when `GRPO_KL_COEF` > 0 and a frozen `ref_net` exists): $+\beta \widehat{\text{KL}}(\pi \parallel \pi_{\text{ref}})$ with the unbiased k3 estimator, gradient flowing through $\log \pi$. It is logged in the `vf` slot (no critic) so the heartbeat prints unchanged;
- **[B2] aux value** (when `GRPO_AUX_VALUE`): an MSE of the (otherwise unused) value head to the MC return, weighted by `GRPO_AUX_COEF`, as a shared-trunk regularizer — it does *not* enter the critic-free advantage and keeps the weights PPO-compatible.
- **aux head** (when `AUX_WIN_BET` or `AUX_REWARD_PRED`): the same shallow scalar head's win-bet ($-b_t z$, $b_t = \tanh$) or reward-prediction (MSE) loss as PPO (document 2) is added here too, so the critic-free path — including `ALGO="mc"` — trains it (representation aux only).

5 Empirical verdict: the estimators failed; legacy is the keeper

The original symptom. Legacy GRPO (`GRPO_STD_NORM=True`) was observed to death-spiral in self-play (it426→it526: launch/step 0.88 → 0.37, learner elo2 2124 → 1772, losing to its own earlier snapshot and to the `greedy` watchdog). The advantage-bound probe (`grpо_diag.advantage_bound_probe`) showed that as the snapshot pool strengthens, the learner faces *lopsided losing groups* where a rare win's per-group-std advantage is $\sqrt{(1-p)/p}$ — unbounded as $p \rightarrow 0$. The hypothesis was that

the policy chases those lucky wins into a passivity ratchet, and that a *bounded, scale-invariant* estimator (rank) would fix it.

The controlled result. Three matched 3070 Ti A/Bs (`scripts/_gpu_ab_anticollapse.py`, `_gpu_ab2.py`, `_gpu_ab3.py`; h256 trunk, BC-warm, 2p, 100 iters, shared init/seed) reversed the hypothesis:

config	launch/step	elo2	verdict
legacy std-norm	0.70 → 0.59	1497 → 1576	stable / improving
rank [C1] + clip (the shipped default!)	0.53 → 0.10	→ 1259	passivity collapse
rank-noclip	0.48 → 0.27	→ 1292	passivity
Dr.GRPO [B] center-only	0.57 → 0.34	→ 1370	passivity
Dr.GRPO + clip-hi	0.46 → 0.03	→ 1274	worst
sibling [B1] (an “all-on” path)	0.57 → 0.24	→ 1295	passivity ($ A _{\max}$ 16)
phi-value [C] (the other “all-on” path)	0.59 → 0.31	→ 1358	passivity ($ A _{\max}$ 22)

The cause, and a footgun.

1. **The collapse is the estimator change itself.** Every Tier 0–2 estimator (rank, with or without clip; Dr.GRPO center-only; sibling; phi-value) drives launch-rate and Elo down, while the legacy per-group std-norm baseline is stable and improving on the SAME shared init. The reward decay is *not* a factor: an earlier hypothesis that an asymmetric “lose-slower” decay triggered the collapse did not hold up.
2. **The “all flags on” A100 collapse was also a precedence footgun.** With every flag on, `grpo_adv_mode` selects sibling [B1] or phi-value [C] — never rank — and those per-step paths both go passive *and* throw heavy-tailed advantages ($|A|_{\max}$ 16–22). The guard (active-estimator log + ignored-estimator warning) was added so this is visible.

Why “bounded $\neq$ good”. The $\sqrt{3}$ rank bound held exactly — but a rank advantage discards win *magnitude*, so it stops rewarding decisive aggression and settles into the passivity basin (the project’s oldest failure mode). The std-norm rare-win amplification the probe flagged is double-edged: it is also what *keeps* the agent aggressive. Removing the amplifier removed the aggression.

The keeper, and the real next lever. Train GRPO with

`GRPO_STD_NORM=True`, every estimator flag off, `CLIP_HI=0`

If the `it426`-style spiral recurs under the keeper, the evidence points to a **trust-region / entropy** lever (KL-to-reference `GRPO_KL_COEF` > 0, or an entropy floor) on top of legacy std-norm — *not* an advantage-estimator swap. The meta-lesson, now in the project memory (`grpo-anticollapse-v12`): a deterministic proof of a *property* (boundedness) is not a proof of *improving the objective*; always local-ablate on the 3070 Ti before spending A100 time.

Diagnostic harness. `grpo_diag.py` holds `advantage_bound_probe` (deterministic CPU bound + difficulty-inversion check), `run_anticollapse_compare` (the 3-arm A/B, with `resume_from` to reproduce the real spiral on GPU), and `plot_anticollapse`. The `ANTICOLLAPSE / DRGRPO_BUNDLE` dicts in that module are a *research harness that empirically failed*, not a recommended configuration.

The rollout exposes `adv_absmax` / `adv_std` (and an `aMx` heartbeat field) so a heavy-tailed advantage is visible live.

6 Config-knob map

Estimator flags (mutually exclusive; **all default off = the keeper**):

field	default	estimator
GRPO_STD_NORM	True	[B] per-group std-norm (keeper; False = Dr.GRPO center-only)
GRPO_ANALYTIC_ADV	False	[A] 2p closed-form (not anti-collapse)
GRPO_RANK_ADV	False	[C1] bounded average-rank — <i>went passive</i>
GRPO_CVAR_ALPHA	0.0	[C2] worst- α tail baseline
GRPO_SIBLING_BASELINE	False	[B1] per-step matched-prefix sibling GAE
GRPO_PHI_VALUE	False	[C] Φ -as-value per-step GAE

Group / orthogonal knobs:

field	default	controls
ALGO	"ppo"	set to " grpo " to select this learner
GRPO_GROUP	0 (=GROUP_SIZE)	rollouts sharing one world
GRPO_OUTCOME_ONLY	False	baseline on the terminal outcome only
GRPO_LOO	False	leave-one-out group mean (RLOO)
GRPO_KL_COEF	0.0	KL-to-reference β (the recommended next lever)
GRPO_AUX_VALUE	False	[B2] value-head MC-regression representation aux
ADV_TARGET_RMS	0.0	[D4] pin the advantage RMS
CLIP_HI	0.0	[D3] asymmetric upper clip (keeper = 0)
GRPO_CRN	False	[D] common-random-numbers opponent coupling